Podcast 20 — NoSQL & Specialty Databases

Podcast 20 — NoSQL & Specialty Databases

Length target: 15 min **Repo references**: `domain-2-new-solutions/11-dynamodb.md`, `12-elasticache.md`, `13-timestream.md`, `34-neptune-keyspaces.md`, `38-documentdb.md`

Topic & Scope

DynamoDB deserves deep coverage. ElastiCache (Redis/Valkey/Memcached) is heavily tested for caching patterns. Niche DBs (Neptune, Keyspaces, Timestream, DocumentDB) need "what + when" recognition only.

Service Coverage Depth

Deep: - DynamoDB — provisioned vs on-demand, GSI vs LSI, Streams, TTL, transactions, Global Tables, DAX - ElastiCache — Redis / Valkey vs Memcached, cluster mode, snapshotting, encryption

Brief — "what + when": - Neptune (graph database) - Keyspaces (managed Cassandra) - Timestream (time-series) - DocumentDB (MongoDB-compatible — covered in podcast 19)

Structured Outline

1. **Open (30s)** — "DynamoDB and ElastiCache dominate this podcast. The niche DBs need a one-liner each."
2. **DynamoDB Deep (6 min)** — partition key + sort key, capacity modes (provisioned with Auto Scaling vs on-demand), GSI vs LSI, Streams + Lambda, TTL, Transactions, Global Tables (multi-master), DAX (in-memory cache), backup (continuous PITR vs on-demand)
3. **ElastiCache (4 min)** — Redis/Valkey (replication, persistence, pub/sub, sorted sets) vs Memcached (sharding, multithreaded, no persistence), cluster mode, MemoryDB for Redis (durable Redis-compatible primary DB)
4. **Specialty DBs Quick Round (3 min)** — Neptune (graph: property + RDF, Gremlin + SPARQL), Keyspaces (managed Cassandra, CQL), Timestream (time-series, IoT/metrics with auto-tiering), DocumentDB (MongoDB API)
5. **Caching Patterns (1 min)** — Lazy loading vs Write-through vs TTL invalidation
6. **Rapid-Fire Trap Drill (60s)**

Must-Mention Exam Tips

- DynamoDB capacity: provisioned (set RCU/WCU + Auto Scaling) vs on-demand (pay per request, scales infinitely)
- DynamoDB partition key drives sharding — design for even access
- DynamoDB sort key enables range queries within a partition

- GSI: any attribute as key, separate capacity, eventual consistency, can add anytime
- LSI: same partition key, alternate sort key, must create at table creation, max 5
- DynamoDB Streams: change log of items (24h retention), feeds Lambda for CDC
- TTL: automatic item expiration based on epoch timestamp attribute
- Transactions: ACID across multiple items / tables — costs 2× the capacity
- Global Tables: active-active multi-Region, multi-master
- DAX: managed cluster cache, microsecond reads, write-through
- Continuous backup with PITR: 35-day window, restore to any second
- ElastiCache Redis: replication, snapshots, pub/sub, Lua, persistence (RDB/AOF) — single-threaded core
- ElastiCache Valkey: open-source Redis fork, drop-in replacement, AWS-supported going forward
- ElastiCache Memcached: multi-threaded, sharded, NO persistence, NO replication
- MemoryDB for Redis: durable Redis with multi-AZ transaction log, can serve as primary database
- Neptune: graph DB, supports property graph (Gremlin) + RDF (SPARQL), used for social/fraud/recommendations
- Keyspaces: serverless managed Apache Cassandra (CQL API)
- Timestream: time-series, automatic tiering (memory + magnetic), built-in functions (interpolation, smoothing)
- DocumentDB: MongoDB API compatibility (covered in podcast 19)

Must-Mention Exam Traps

- EXAM TRAP: DynamoDB on-demand vs provisioned — on-demand has no scaling latency but costs more per request at sustained load
- EXAM TRAP: DynamoDB hot partition = throttling — design partition key for even distribution
- EXAM TRAP: GSI has its OWN capacity — must size separately
- EXAM TRAP: LSI must be created at table creation — cannot add later
- EXAM TRAP: DynamoDB transactions DOUBLE the capacity cost — use sparingly
- EXAM TRAP: DynamoDB Streams retention is 24 hours — for longer, fan out to Kinesis Data Streams via the new “Streams to Kinesis” option
- EXAM TRAP: Global Tables need streams enabled — and they replicate ALL items
- EXAM TRAP: DAX is DynamoDB-specific — not a general cache
- EXAM TRAP: ElastiCache Memcached has NO persistence — data loss on restart
- EXAM TRAP: ElastiCache Redis is single-threaded for core operations — CPU-bound
- EXAM TRAP: MemoryDB ≠ ElastiCache — MemoryDB is durable (primary DB tier); ElastiCache is cache tier
- EXAM TRAP: Neptune supports Gremlin OR SPARQL — not arbitrary SQL
- EXAM TRAP: Keyspaces is serverless — no instances to manage; capacity modes like DynamoDB
- EXAM TRAP: Timestream uses memory + magnetic store tiers — older data auto-moves
- EXAM TRAP: Caching strategies: lazy loading misses on first read; write-through doubles writes; pick per workload

Key Decision Matrix

Scenario	Pick
Massive-scale key-value, single-digit ms	DynamoDB
Unpredictable spiky DynamoDB workload	On-demand mode
Steady predictable DynamoDB load	Provisioned + Auto Scaling
Multi-Region active-active KV	DynamoDB Global Tables
Microsecond DynamoDB reads	DAX
Distributed cache, advanced data structures	ElastiCache Redis/Valkey
Simple cache, sharded multithreaded	ElastiCache Memcached
Durable Redis-compatible primary DB	MemoryDB for Redis
Graph relationships, social, fraud	Neptune
Apache Cassandra workload, managed	Keyspaces
Time-series, IoT, metrics auto-tiering	Timestream
MongoDB-compatible managed	DocumentDB

Tone & Style

- Lead with DynamoDB (the heavy hitter)
- For ElastiCache, frame as “Redis = features, Memcached = simple sharding”
- Niche DBs get a sentence each — “When you see graph relationships, pick Neptune”

Rapid-Fire Closer

“DynamoDB hot partition?” — “Throttling.” “DAX cache?” — “DynamoDB only.” “Graph DB?” — “Neptune.” “Cassandra-compatible?” — “Keyspaces.” “Time-series IoT?” — “Timestream.” “Durable Redis primary?” — “MemoryDB.”